
RAPPORT DE STAGE DE FIN D'ANNEE

MotoRaid

Conception et développement d'un écosystème numérique (SaaS) dédié aux motards au Maroc

Présenté par

Hiba OUAHMANE

Encadré par

Mme wafae

Période de stage

Du 01 /07 /2026 au 15 / 08 / 2026

Année universitaire : 2025/ 2026

Remerciements

Au terme de ce stage de fin d'études, je tiens à exprimer ma sincère gratitude envers toutes les personnes qui, de près ou de loin, ont contribué à la réussite de ce projet intitulé « MotoRaid ».

Je souhaite tout d'abord remercier mon encadrante de stage **madame wafae et aussi mr aymane** pour sa disponibilité, ses conseils avisés, sa patience et la confiance qu'il m'a accordée tout au long de cette expérience. Ses remarques constructives et son exigence en matière de qualité technique m'ont permis de progresser considérablement, tant sur le plan des compétences techniques que sur celui de la rigueur méthodologique.

Mes remerciements s'adressent également à mon encadrant pédagogique ainsi qu'à l'ensemble du corps professoral de mon établissement de formation, pour la qualité de l'enseignement dispensé durant mon cursus, lequel m'a fourni les bases théoriques et pratiques indispensables à la réalisation de ce projet.

Je tiens aussi à remercier l'ensemble de l'équipe technique et administrative de l'organisme d'accueil pour leur accueil chaleureux, leur esprit collaboratif et le partage de leur expérience professionnelle, qui ont grandement enrichi ma compréhension du monde de l'entreprise et des bonnes pratiques du développement logiciel.

Enfin, je n'oublie pas de remercier ma famille et mes proches pour leur soutien moral indéfectible, ainsi que toute personne ayant, d'une manière ou d'une autre, contribué à l'aboutissement de ce travail.

Sommaire

La table des matières ci-dessous sera générée automatiquement. Dans Microsoft Word, faites un clic droit sur le champ grisé puis sélectionnez « Mettre à jour les champs » (ou appuyez sur F9) afin d'actualiser la pagination une fois la mise en forme finale du document terminée.

Remerciements	2
Sommaire	3
Introduction Générale	5
Contexte et problématique	5
Objectifs du projet	5
Structure du rapport	6
Chapitre 1 : Analyse et Conception	7
1.1. Cahier des charges	7
Rôles identifiés	7
Fonctionnalités attendues par rôle	7
1.2. Modélisation UML	7
Diagramme de cas d'utilisation global	7
Explication des cas d'utilisation principaux	9
Diagramme de classes (base de données)	9
Explication du modèle relationnel et de la gestion du stock	9
Chapitre 2 : Architecture et Technologies	11
2.1. Choix techniques	11
Backend : Laravel 11 (PHP 8.2)	11
Frontend : Vue.js 3 (Composition API) et Tailwind CSS	11
Couche de liaison : Inertia.js	11
Base de données : SQLite	11
2.2. Architecture logicielle	12
Extension du MVC par des Service Classes	12
Schéma de l'architecture MVC/Inertia	12
Sécurité applicative	13
Chapitre 3 : Réalisation et Développement	14
3.1. La Marketplace de Location (Rentals)	14
Gestion de la flotte par l'Admin/Partenaire	14
Calcul de disponibilité (chevauchement de dates)	14
Parcours utilisateur (modale de réservation, accessoires)	14
Intégration du paiement simulé (PayPal) et génération de facture PDF	15
3.2. Le Planificateur GPS Avancé	15
Intégration de Leaflet et OpenStreetMap	15

Routage dynamique (OSRM) et mode « Éviter les autoroutes »	15
API Météo (Open-Meteo) et hébergements.....	16
Export au format GPX	16
3.3. Réseau Social & Ride Hubs	16
Création de groupes, chat en direct, système de validation	16
3.4. Sécurité et Tracking Live	17
Tracking en temps réel (Polling)	17
Système SOS (middleware, notifications, e-mail d'urgence)	17
Diagramme de séquence du déclenchement SOS	18
3.5. Expérience Utilisateur (UX/UI) et Monétisation	19
Design « Dark Mode / Glassmorphism »	19
Dashboard Admin dynamique (Chart.js)	19
Système d'abonnement Freemium et cloche de notifications	19
Chapitre 4 : Tests et Déploiement.....	20
4.1. Stratégie de test.....	20
Tests manuels	20
Seeders pour les données de démonstration	20
4.2. Déploiement	20
Préparation du livrable	20
Discussion sur l'évolution mobile future (Capacitor.js)	21
Conclusion Générale	22
Bilan technique et personnel	22
Perspectives d'évolution	22
Bibliographie / Webographie	24

Introduction Générale

Contexte et problématique

Le secteur du deux-roues motorisé connaît au Maroc une croissance continue, portée par un tissu de motards de plus en plus structuré, actif sur les réseaux sociaux et friand d'événements collectifs tels que les « rides » régionaux, les sorties de week-end ou les raids longue distance. Cette communauté, bien que dynamique, demeure aujourd'hui fragmentée : la location de motos repose largement sur des échanges informels via des groupes Facebook ou WhatsApp, la planification d'itinéraires s'effectue à l'aide d'outils génériques peu adaptés aux spécificités de la conduite moto (relief, qualité de la chaussure, contournement des autoroutes), et la sécurité des motards en déplacement, notamment lors des sorties en groupe ou en solo sur de longues distances, ne bénéficie d'aucun dispositif de suivi ou d'alerte dédié.

C'est de ce constat qu'est né le projet MotoRaid : la volonté de proposer à la communauté marocaine des motards un écosystème numérique unique, centralisé et pensé spécifiquement pour leurs besoins, en rupture avec la juxtaposition d'outils disparates et non interopérables utilisés jusqu'alors. Le projet se positionne à la croisée de plusieurs problématiques : celle du commerce en ligne appliqué à la location de véhicules (marketplace B2B2C), celle de la cartographie et du routage appliqués à la moto, celle du réseau social communautaire, et enfin celle de la sécurité connectée, avec un système de suivi en temps réel et d'alerte d'urgence.

La problématique centrale à laquelle ce stage a tenté de répondre peut être formulée ainsi : comment concevoir et développer, avec des ressources et un délai de stage limités, une plateforme SaaS robuste, sécurisée et ergonomique, capable de couvrir l'ensemble du parcours du motard marocain — de la découverte d'une moto à louer jusqu'à son retour en toute sécurité — tout en respectant les bonnes pratiques d'architecture logicielle qui garantissent l'évolutivité et la maintenabilité du produit ?

Objectifs du projet

Le projet MotoRaid poursuit plusieurs objectifs, à la fois fonctionnels et techniques :

- Concevoir une marketplace de location de motos permettant à des partenaires (loueurs professionnels) et à l'administrateur de la plateforme de gérer leur flotte, avec une gestion automatisée de la disponibilité et du stock ;
- Développer un planificateur d'itinéraires GPS avancé, intégrant cartographie libre, routage dynamique, données météorologiques et export au format standard GPX ;
- Mettre en place un réseau social communautaire (Ride Hubs) permettant aux motards de créer des groupes, d'échanger en temps réel et de valider leur participation à des sorties ;
- Concevoir un système de suivi en temps réel (tracking live) et un bouton d'alerte SOS afin de renforcer la sécurité des utilisateurs lors de leurs déplacements ;
- Fournir un tableau de bord administrateur complet permettant le pilotage global de la plateforme (utilisateurs, réservations, flotte, statistiques) ;
- Explorer l'intégration d'un assistant intelligent (IA) en support à l'utilisateur ;
- Mettre en œuvre un modèle de monétisation via un système d'abonnement Freemium et une passerelle de paiement en ligne (PayPal) ;

- Appliquer, tout au long du développement, les principes d'architecture logicielle SOLID et les bonnes pratiques du Génie Logiciel, afin de produire un code maintenable, testable et évolutif.

Structure du rapport

Le présent rapport est organisé en quatre chapitres principaux, précédés de cette introduction générale et suivis d'une conclusion. Le premier chapitre est consacré à l'analyse des besoins et à la conception du système, à travers l'élaboration du cahier des charges et la modélisation UML. Le deuxième chapitre présente l'architecture logicielle retenue ainsi que les choix technologiques qui la sous-tendent. Le troisième chapitre, le plus conséquent, détaille la réalisation technique de chacun des modules fonctionnels de la plateforme. Le quatrième chapitre aborde la stratégie de test adoptée ainsi que les modalités de déploiement du livrable. Enfin, le rapport se conclut par un bilan général du stage et par une ouverture sur les perspectives d'évolution du projet.

Chapitre 1 : Analyse et Conception

Ce premier chapitre pose les fondations du projet MotoRaid. Il expose dans un premier temps le cahier des charges fonctionnel, en définissant précisément les rôles des utilisateurs et les fonctionnalités qui leur sont accessibles, puis propose une modélisation UML du système permettant d'en formaliser la structure et le comportement avant toute phase de codage.

1.1. Cahier des charges

La définition du périmètre fonctionnel de MotoRaid repose sur une analyse fine des différents profils d'utilisateurs appelés à interagir avec la plateforme. Quatre rôles distincts ont été identifiés, chacun disposant d'un niveau d'accès et de responsabilités propres, conformément à une logique de gestion des droits par rôle (Role-Based Access Control, RBAC).

Rôles identifiés

Rôle	Description
Visiteur / User	Utilisateur inscrit standard : consultation du catalogue, réservation de motos, planification d'itinéraires, participation aux Ride Hubs.
Utilisateur Premium	Utilisateur ayant souscrit à un abonnement payant, bénéficiant de fonctionnalités avancées (tracking étendu, assistant IA, options du planificateur).
Administrateur	Pilotage global de la plateforme : gestion des utilisateurs, des partenaires, de la flotte globale, supervision du tracking et des alertes SOS, statistiques.

Fonctionnalités attendues par rôle

Le tableau suivant synthétise, de manière non exhaustive, les principales fonctionnalités attendues pour chaque rôle identifié :

Rôle	Fonctionnalités principales
User	Recherche et réservation de motos, planification d'itinéraires, participation aux Ride Hubs, activation du SOS, notifications.
Premium	Toutes les fonctionnalités User + assistant IA, options avancées du planificateur, badge et statut visibles sur le profil.
Admin	Supervision globale, gestion des rôles, modération des Ride Hubs, dashboard statistique, gestion des alertes SOS.

1.2. Modélisation UML

Afin de formaliser la compréhension du système avant son implémentation, plusieurs diagrammes UML (Unified Modeling Language) ont été élaborés. Ils constituent un support de communication essentiel avec l'encadrant de stage et facilitent la traçabilité entre les besoins exprimés et les choix d'implémentation réalisés par la suite.

Diagramme de cas d'utilisation global

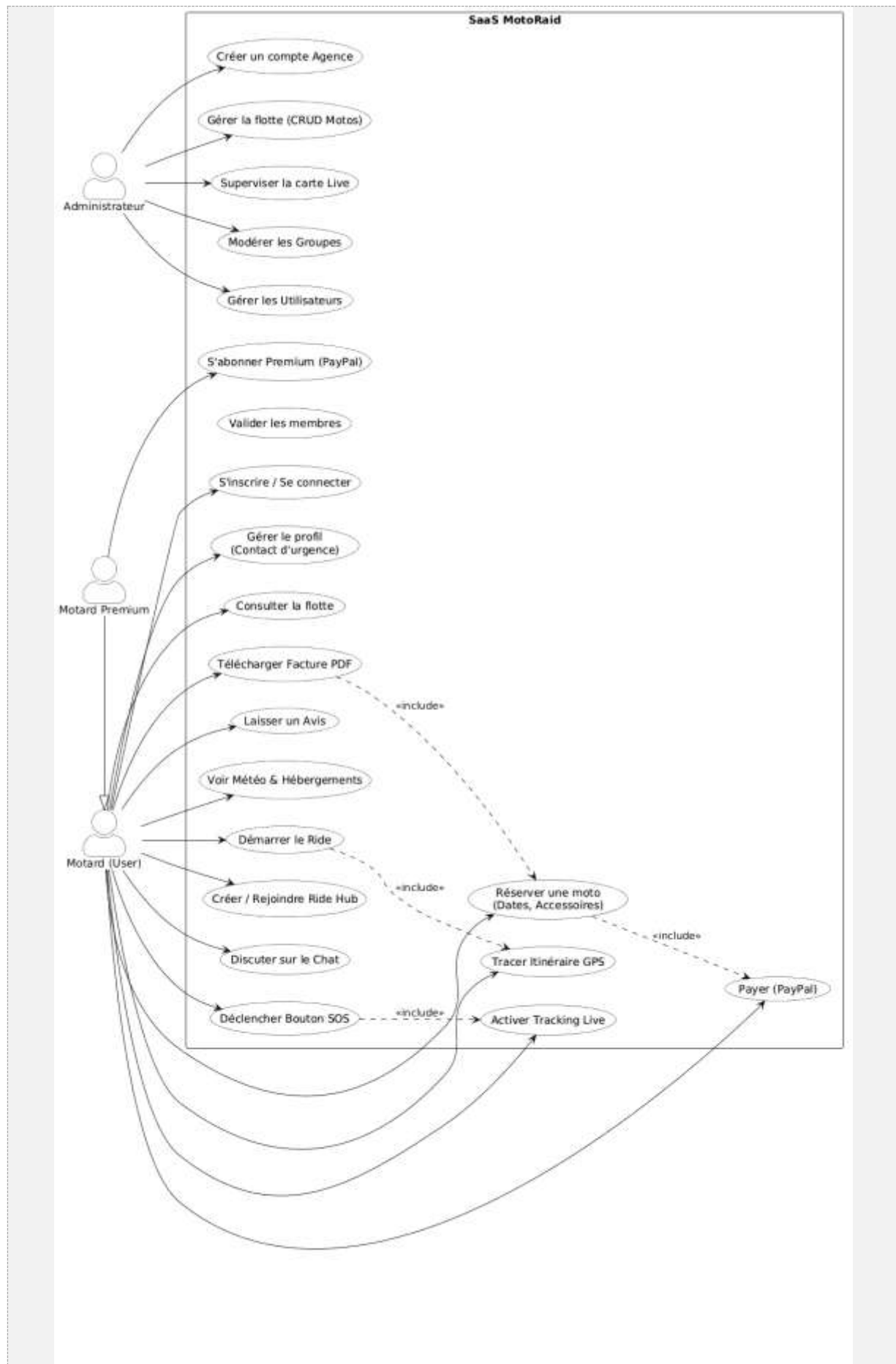


Figure 1 — Diagramme de Cas d'Utilisation global

Ce diagramme représente l'ensemble des interactions possibles entre les différents acteurs du système (Visiteur, User, Premium, Partenaire, Admin) et les principaux cas d'utilisation offerts par la plateforme : « Rechercher une moto », « Réserver une moto », « Planifier un itinéraire », « Créer un Ride Hub », « Déclencher une alerte SOS », « Gérer la flotte », « Consulter le tableau de bord », etc. Les relations d'inclusion (<<include>>) et d'extension (<<extend>>) permettent notamment de modéliser la dépendance entre la réservation d'une moto et la vérification préalable de la disponibilité, ou encore l'extension optionnelle du parcours de réservation par l'ajout d'accessoires.

Explication des cas d'utilisation principaux

- « Réserver une moto » : inclut systématiquement « Vérifier la disponibilité », et peut être étendu par « Ajouter des accessoires » et « Effectuer le paiement » ;
- « Planifier un itinéraire » : inclut « Calculer l'itinéraire (OSRM) » et peut être étendu par « Exporter au format GPX » et « Consulter la météo » ;
- « Déclencher une alerte SOS » : inclut « Notifier le contact d'urgence » et « Notifier l'administrateur » ;
- « Gérer la flotte » (Partenaire/Admin) : inclut « Ajouter une moto », « Modifier une moto » et « Gérer le stock ».

Diagramme de classes (base de données)

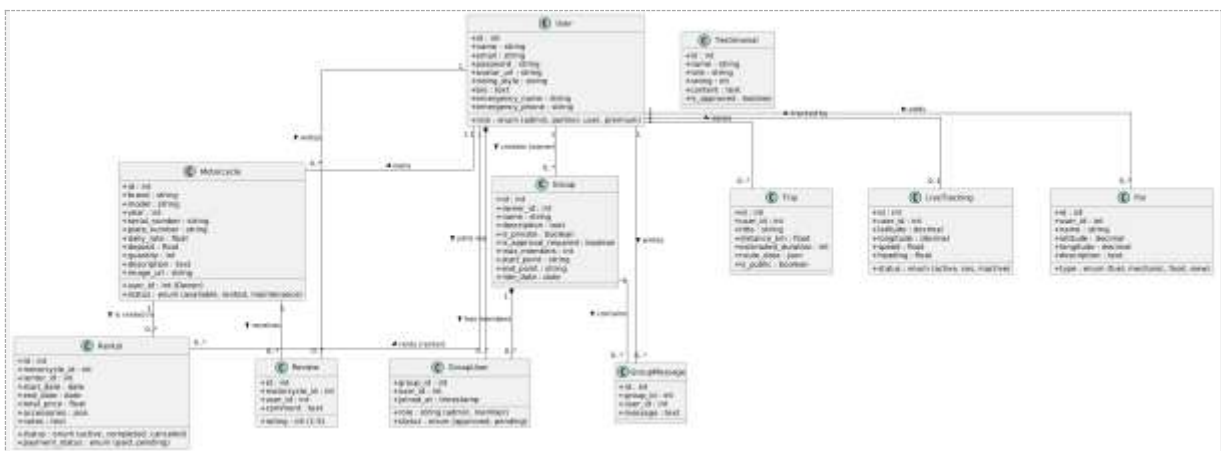


Figure 2 — Diagramme de Classes (Base de données)

Le diagramme de classes ci-dessus modélise la structure relationnelle de la base de données de MotoRaid. Il met en évidence les entités centrales du système — User, Moto, Reservation, RideHub, Message, TrackingSession, AlertSOS, Abonnement — ainsi que les relations de cardinalité qui les unissent. Une attention particulière a été portée à la modélisation de la relation entre Moto et Reservation, laquelle conditionne le calcul dynamique de la disponibilité, ainsi qu'à la relation entre User et RideHub, matérialisée par une table pivot permettant de gérer l'appartenance à plusieurs groupes ainsi que le statut de validation de chaque membre.

Explication du modèle relationnel et de la gestion du stock

La gestion du stock repose sur un principe de vérification de chevauchement de plages de dates (« date overlap »). Chaque moto dispose d'un attribut de quantité disponible en stock ; lorsqu'une

réserve est initiée pour une période donnée, le système interroge l'ensemble des réservations existantes pour cette moto dont les dates chevauchent la période demandée, et en déduit la disponibilité résiduelle. Cette logique, bien que simple en apparence, nécessite une attention particulière afin d'éviter les conditions de concurrence (race conditions) lors de réservations simultanées ; elle a été encapsulée dans une classe de service dédiée, détaillée au chapitre suivant, afin de garantir la cohérence des données en toutes circonstances.

Chapitre 2 : Architecture et Technologies

Ce chapitre expose les choix technologiques opérés pour la réalisation de MotoRaid ainsi que l'architecture logicielle mise en place, laquelle vise à garantir la robustesse, la maintenabilité et l'évolutivité du système.

2.1. Choix techniques

Le projet MotoRaid a été conçu selon une architecture de type « monolithe moderne », choix délibéré permettant de bénéficier de la rapidité de développement propre aux applications monolithiques tout en conservant une organisation interne modulaire, préparant ainsi une éventuelle décomposition future en services indépendants si la charge ou la complexité du produit venait à le justifier.

Backend : Laravel 11 (PHP 8.2)

Le backend de l'application repose sur le framework Laravel, dans sa version 11, exécuté sur PHP 8.2. Ce choix se justifie par la maturité de l'écosystème Laravel, la richesse de ses composants natifs (Eloquent ORM, système de files d'attente, notifications, authentification via Laravel Breeze/Fortify, planification de tâches) et par la qualité de sa documentation, autant d'atouts permettant de se concentrer sur la logique métier plutôt que sur la réinvention d'infrastructures techniques déjà éprouvées.

Frontend : Vue.js 3 (Composition API) et Tailwind CSS

La couche de présentation a été développée avec Vue.js 3, en s'appuyant sur la Composition API, laquelle favorise la réutilisabilité de la logique par le biais de « composables » et améliore la lisibilité du code face à l'ancienne Options API. La mise en forme visuelle repose sur le framework utilitaire Tailwind CSS, choisi pour sa rapidité de mise en œuvre et sa cohérence avec l'approche « design system » adoptée pour l'interface (voir section 3.5).

Couche de liaison : Inertia.js

L'articulation entre le backend Laravel et le frontend Vue.js est assurée par Inertia.js, une bibliothèque permettant de construire des applications monopages (SPA) modernes sans avoir à développer une API REST ou GraphQL distincte. Inertia.js permet ainsi de conserver les avantages du routage et des contrôleurs Laravel côté serveur, tout en offrant une expérience de navigation fluide et réactive côté client, sans rechargement de page. Ce choix architectural réduit significativement la duplication de logique de validation entre le frontend et le backend, et simplifie la gestion de l'état de l'authentification.

Base de données : SQLite

Pour les besoins du développement et de la démonstration du projet, une base de données SQLite locale a été retenue. Ce choix, motivé par la simplicité de configuration qu'il autorise (absence de serveur de base de données dédié) et par sa portabilité (fichier unique embarqué avec le livrable), n'obère en rien la portabilité future de l'application vers un système de gestion de base de données plus robuste tel que MySQL ou PostgreSQL, Laravel Eloquent assurant une abstraction suffisante de la couche de persistance.

2.2. Architecture logicielle

Si le pattern architectural MVC (Modèle-Vue-Contrôleur) constitue le socle natif de Laravel, il s'avère rapidement insuffisant, à lui seul, pour un projet de l'envergure de MotoRaid : en confiant l'intégralité de la logique métier aux contrôleurs, on aboutit inévitablement à des classes volumineuses, difficiles à tester et à faire évoluer, communément désignées sous le terme de « Fat Controllers ».

Extension du MVC par des Service Classes

Afin de pallier cet écueil, l'architecture de MotoRaid introduit une couche supplémentaire de « Service Classes » (classes de service), positionnée entre les contrôleurs et les modèles Eloquent. Chaque domaine fonctionnel majeur — réservation, planification d'itinéraire, gestion du tracking, gestion des alertes SOS, facturation — dispose ainsi de sa propre classe de service, responsable de l'intégralité de la logique métier associée. Le contrôleur se limite alors à un rôle de coordination : réception de la requête HTTP, validation des données via des classes FormRequest dédiées, délégation à la classe de service appropriée, puis retour de la réponse Inertia.

Cette organisation permet de respecter les principes SOLID, pierre angulaire de la conception orientée objet retenue pour le projet :

- Single Responsibility Principle (SRP) : chaque classe de service n'a qu'une seule raison de changer, par exemple ReservationService ne traite que la logique de réservation, indépendamment de la logique de notification ou de facturation ;
- Open/Closed Principle (OCP) : l'ajout d'un nouveau mode de paiement, par exemple, se fait par extension (implémentation d'une nouvelle stratégie) sans modification du code existant ;
- Liskov Substitution Principle (LSP) : les interfaces de service (par exemple une interface PaymentGatewayInterface) garantissent que toute implémentation concrète peut se substituer à une autre sans altérer le comportement attendu ;
- Interface Segregation Principle (ISP) : les contrats d'interface sont volontairement restreints à un ensemble cohérent de méthodes, évitant d'imposer aux classes clientes des dépendances superflues ;
- Dependency Inversion Principle (DIP) : les classes de haut niveau (contrôleurs) dépendent d'abstractions (interfaces) et non d'implémentations concrètes, l'injection de dépendances étant nativement gérée par le conteneur de services de Laravel.

Schéma de l'architecture MVC/Inertia

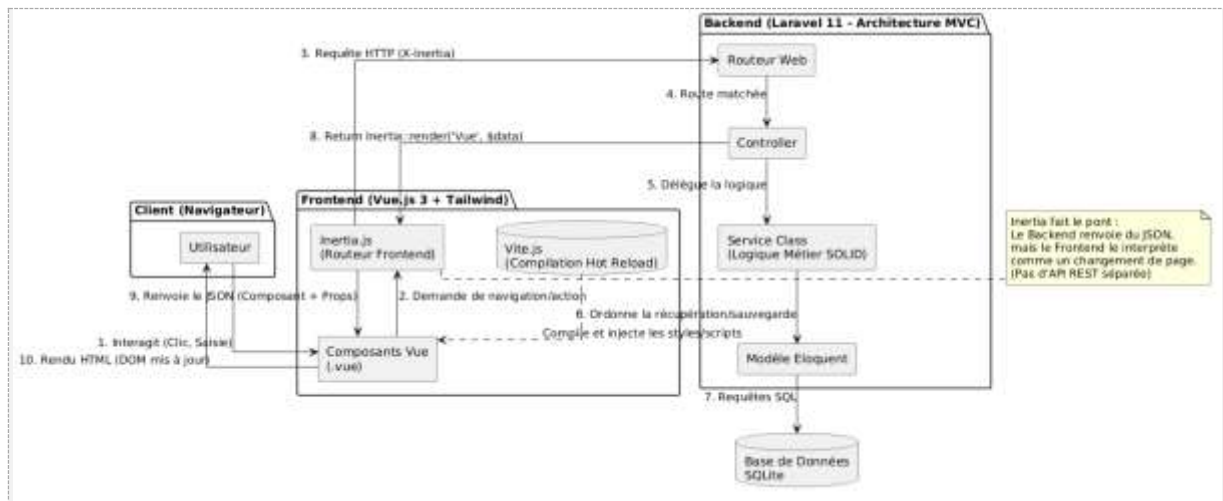


Figure 3 — Schéma de l'architecture MVC/Inertia

Sécurité applicative

La sécurité a été pensée de manière transversale dans l'ensemble de l'architecture. L'authentification et la gestion des sessions s'appuient sur les mécanismes natifs de Laravel, complétés par un système d'autorisations (Policies et Gates) garantissant qu'un utilisateur ne peut accéder qu'aux ressources relevant de son rôle — un partenaire, par exemple, ne pouvant modifier que les motos appartenant à sa propre flotte. La validation des données est systématiquement effectuée côté serveur au moyen de classes FormRequest dédiées, indépendamment des contrôles réalisés côté client, afin de se prémunir contre toute tentative de contournement. Enfin, les données sensibles, telles que les informations relatives aux contacts d'urgence, bénéficient d'un traitement particulier au niveau de la couche de persistance et de la journalisation.

Chapitre 3 : Réalisation et Développement

Ce chapitre constitue le cœur technique du présent rapport. Il détaille, module par module, la réalisation concrète des fonctionnalités de la plateforme MotoRaid, en mettant en lumière tant les choix d'implémentation que les difficultés rencontrées et les solutions retenues.

3.1. La Marketplace de Location (Rentals)

Le module de location constitue le socle économique de la plateforme MotoRaid. Il met en relation, au sein d'une architecture de type B2B2C (Business-to-Business-to-Consumer), les partenaires professionnels propriétaires de flottes de motos et les utilisateurs finaux souhaitant louer un véhicule pour une sortie ou un raid.

Gestion de la flotte par l'Admin/Partenaire

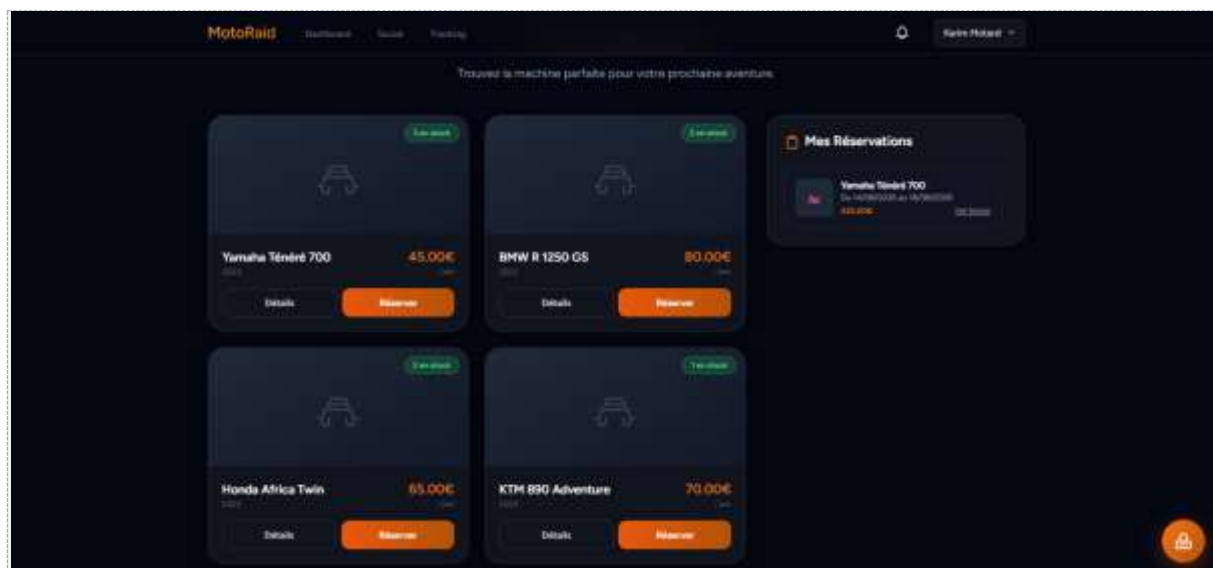
Chaque partenaire dispose d'un espace de gestion dédié lui permettant d'ajouter une nouvelle moto à son catalogue, en renseignant l'ensemble des caractéristiques pertinentes (marque, modèle, cylindrée, année, tarif journalier, photographies, description, équipements inclus) ainsi que la quantité disponible en stock. Le téléversement (upload) des images est géré côté serveur avec redimensionnement et validation du type MIME, afin de garantir tant la sécurité que la performance d'affichage du catalogue. L'administrateur, quant à lui, dispose d'une vision consolidée de l'ensemble des flottes, tous partenaires confondus, et peut intervenir en cas de besoin (modération, suspension d'une annonce non conforme).

Calcul de disponibilité (chevauchement de dates)

La vérification de la disponibilité d'une moto pour une période de location donnée constitue l'un des algorithmes les plus critiques du module. Elle repose sur une requête d'exclusion des réservations dont les dates de début et de fin chevauchent la période sollicitée par l'utilisateur, selon la logique classique de détection d'intervalles superposés : deux intervalles se chevauchent dès lors que la date de début de l'un est antérieure à la date de fin de l'autre, et réciproquement. Cette vérification est encapsulée au sein de la classe de service ReservationService, ce qui permet de la réutiliser de manière cohérente à la fois lors de l'affichage du calendrier de disponibilité côté frontend et lors de la validation finale de la réservation côté serveur, évitant ainsi toute divergence entre ce qui est affiché à l'utilisateur et ce qui est effectivement autorisé par le système.

Parcours utilisateur (modale de réservation, accessoires)

Le parcours de réservation a été pensé pour minimiser la friction rencontrée par l'utilisateur. Depuis la fiche détaillée d'une moto, celui-ci ouvre une fenêtre modale de réservation, dans laquelle il sélectionne ses dates de location via un composant de calendrier interactif reflétant en temps réel la disponibilité calculée côté serveur. L'utilisateur peut ensuite enrichir sa réservation par l'ajout d'accessoires optionnels (casque supplémentaire, top-case, gants), chacun disposant de son propre tarif, avant de valider la commande. Un récapitulatif dynamique du montant total, mis à jour à chaque interaction, est affiché tout au long du parcours.



Capture d'écran 4 — Page de location avec mode de réservation

Intégration du paiement simulé (PayPal) et génération de facture PDF

La validation finale d'une réservation déclenche une intégration avec la passerelle de paiement PayPal, mise en œuvre en environnement de test (sandbox) pour les besoins de la démonstration du projet. Une fois le paiement confirmé, une facture au format PDF est générée automatiquement côté serveur, reprenant l'ensemble des informations de la transaction (identité du locataire, détails de la moto, période de location, accessoires sélectionnés, montant total, numéro de transaction), puis mise à disposition de l'utilisateur au sein de son espace personnel et transmise par courrier électronique.

3.2. Le Planificateur GPS Avancé

Le planificateur d'itinéraires constitue l'une des fonctionnalités différenciantes de MotoRaid, spécifiquement conçue pour répondre aux besoins particuliers de la conduite moto, distincts de ceux de l'automobiliste classique.

Intégration de Leaflet et OpenStreetMap

La cartographie interactive repose sur la bibliothèque JavaScript Leaflet, couplée aux données cartographiques libres d'OpenStreetMap. Ce choix technologique, alternatif aux solutions propriétaires telles que Google Maps, a été retenu pour son absence de coût de licence à l'usage, la richesse de son écosystème de plugins, et la liberté qu'il offre quant à la personnalisation du rendu cartographique, un critère important pour l'identité visuelle de la plateforme.

Routage dynamique (OSRM) et mode « Éviter les autoroutes »

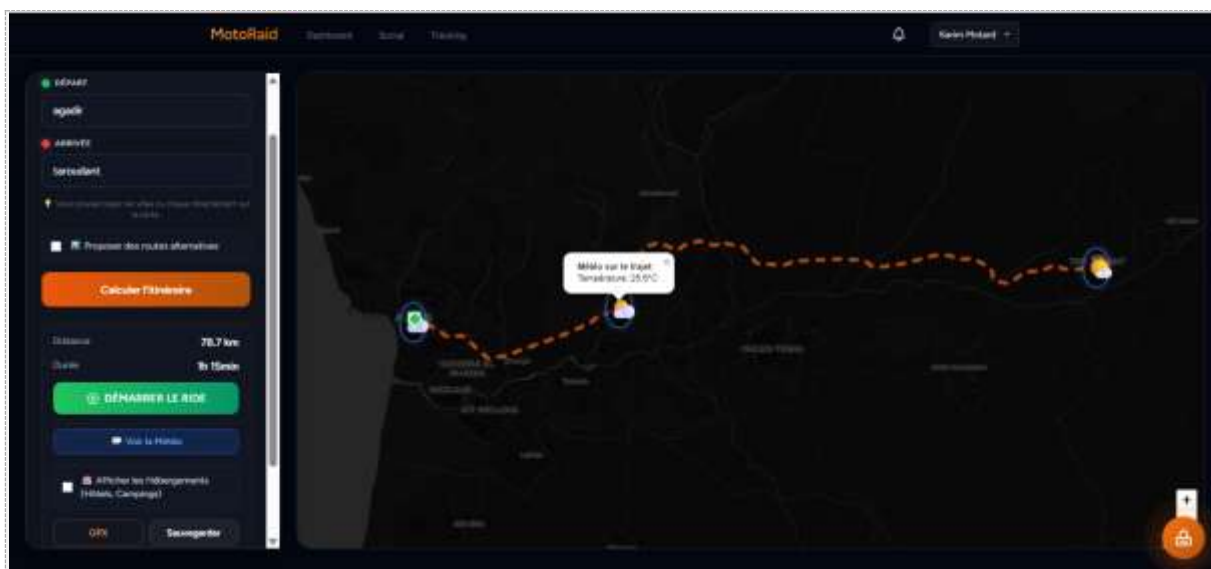
Le calcul d'itinéraire s'appuie sur le moteur de routage open-source OSRM (Open Source Routing Machine), interrogé via son API HTTP depuis le backend Laravel. Une attention particulière a été portée à l'implémentation d'un mode « Éviter les autoroutes », particulièrement pertinent pour un public de motards recherchant davantage des itinéraires sinueux et paysagers que des trajets purement optimisés en temps de parcours. Ce mode est traduit, au niveau de la requête adressée à OSRM, par l'exclusion des profils de route correspondant aux axes autoroutiers, exploitant les paramètres d'exclusion de classes de voies proposés par le moteur de routage.

API Météo (Open-Meteo) et hébergements

Afin d'enrichir la préparation d'un itinéraire, le planificateur intègre une consultation en temps réel des conditions météorologiques prévues le long du trajet, via l'API gratuite Open-Meteo. Ces données sont superposées à la carte sous forme d'indicateurs visuels aux points clés de l'itinéraire (température, risque de précipitations, vent), permettant au motard d'anticiper d'éventuelles conditions de conduite dégradées. Une fonctionnalité complémentaire de recherche d'hébergements à proximité des étapes de l'itinéraire a également été intégrée, particulièrement utile dans le cadre de raids de plusieurs jours.

Export au format GPX

Une fois l'itinéraire finalisé, l'utilisateur peut l'exporter au format GPX (GPS Exchange Format), un format standard largement supporté par les dispositifs de navigation GPS embarqués sur moto ainsi que par les principales applications de navigation mobile. Cette fonctionnalité assure l'interopérabilité de MotoRaid avec l'écosystème matériel existant des motards, qui peuvent ainsi préparer leur itinéraire depuis la plateforme puis le charger directement sur leur propre équipement de navigation.



Capture d'écran 5 — Planificateur GPS avec carte et météo

3.3. Réseau Social & Ride Hubs

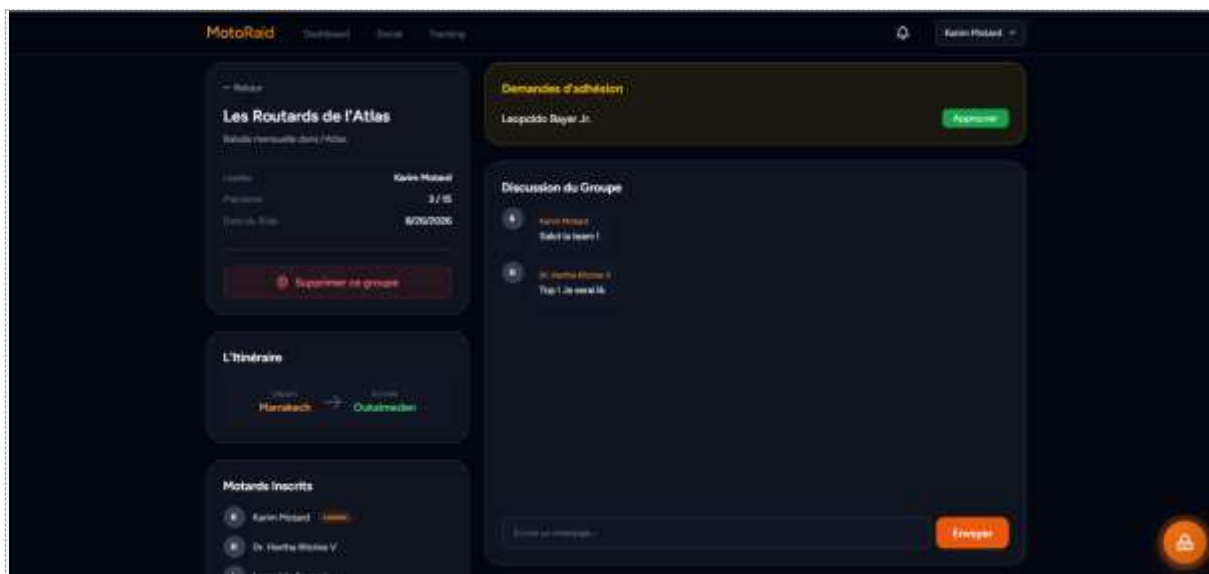
Le volet communautaire de MotoRaid, dénommé « Ride Hubs », répond au besoin identifié en introduction de fédérer les motards autour de sorties organisées, en substituant à l'usage de groupes de réseaux sociaux génériques un espace dédié, mieux intégré au reste des fonctionnalités de la plateforme (planificateur d'itinéraire partagé, tracking en groupe).

Création de groupes, chat en direct, système de validation

Tout utilisateur peut créer un Ride Hub, en définissant son thème (type de sortie, région, niveau de difficulté), sa date et, le cas échéant, l'itinéraire associé, préalablement préparé via le planificateur GPS. Les autres utilisateurs peuvent demander à rejoindre ce groupe, l'organisateur disposant d'un système de validation des demandes d'adhésion afin de conserver la maîtrise de la composition du groupe. Une fois membre, l'utilisateur accède à un espace de discussion en direct dédié au Ride Hub,

permettant la coordination logistique de la sortie (point de rendez-vous, horaires, ajustements de dernière minute).

Sur le plan technique, la messagerie en temps réel a été implémentée en s'appuyant sur un mécanisme de rafraîchissement périodique côté client (voir également la discussion relative aux WebSockets en perspective d'évolution, section « Conclusion Générale »), garantissant une expérience réactive tout en conservant une architecture serveur simple à déployer, en cohérence avec les contraintes du projet.



Capture d'écran 6 — Page du Ride Hub avec Chat

3.4. Sécurité et Tracking Live

Ce module constitue le cœur de la valeur ajoutée de MotoRaid par rapport aux outils existants sur le marché marocain : il matérialise l'engagement de la plateforme envers la sécurité de ses utilisateurs, préoccupation centrale et récurrente exprimée par la communauté des motards, notamment lors de déplacements en solo ou sur de longues distances.

Tracking en temps réel (Polling)

Lorsqu'un utilisateur démarre une session de suivi, sa position géographique, obtenue via l'API de géolocalisation du navigateur ou de l'application, est transmise périodiquement au serveur selon un mécanisme de polling (interrogation régulière à intervalle défini). Cette approche, plus simple à mettre en œuvre et à sécuriser qu'une connexion persistante de type WebSocket dans le contexte d'un stage à durée limitée, permet néanmoins d'obtenir un suivi suffisamment réactif pour les besoins de l'application, tout en maîtrisant la charge serveur générée. Les positions successives sont historisées, permettant de reconstituer a posteriori le tracé complet du trajet effectué.

Système SOS (middleware, notifications, e-mail d'urgence)

Le bouton SOS, accessible en permanence depuis l'interface mobile de l'application durant une session active, permet à l'utilisateur de déclencher une alerte d'urgence en un geste. Ce déclenchement met en œuvre une chaîne de traitement rigoureuse : un middleware dédié intercepte la requête afin d'en vérifier l'authenticité et l'origine, avant de déléguer le traitement à une classe de service SosService. Celle-ci enregistre l'alerte en base de données avec horodatage et dernière position connue, déclenche

une notification immédiate à destination du tableau de bord de l'administrateur (pour une intervention ou un suivi humain), et transmet simultanément un courrier électronique d'urgence au contact préalablement désigné par l'utilisateur dans son profil, contenant la position géographique au moment du déclenchement ainsi qu'un lien vers la carte de suivi en direct.

Diagramme de séquence du déclenchement SOS

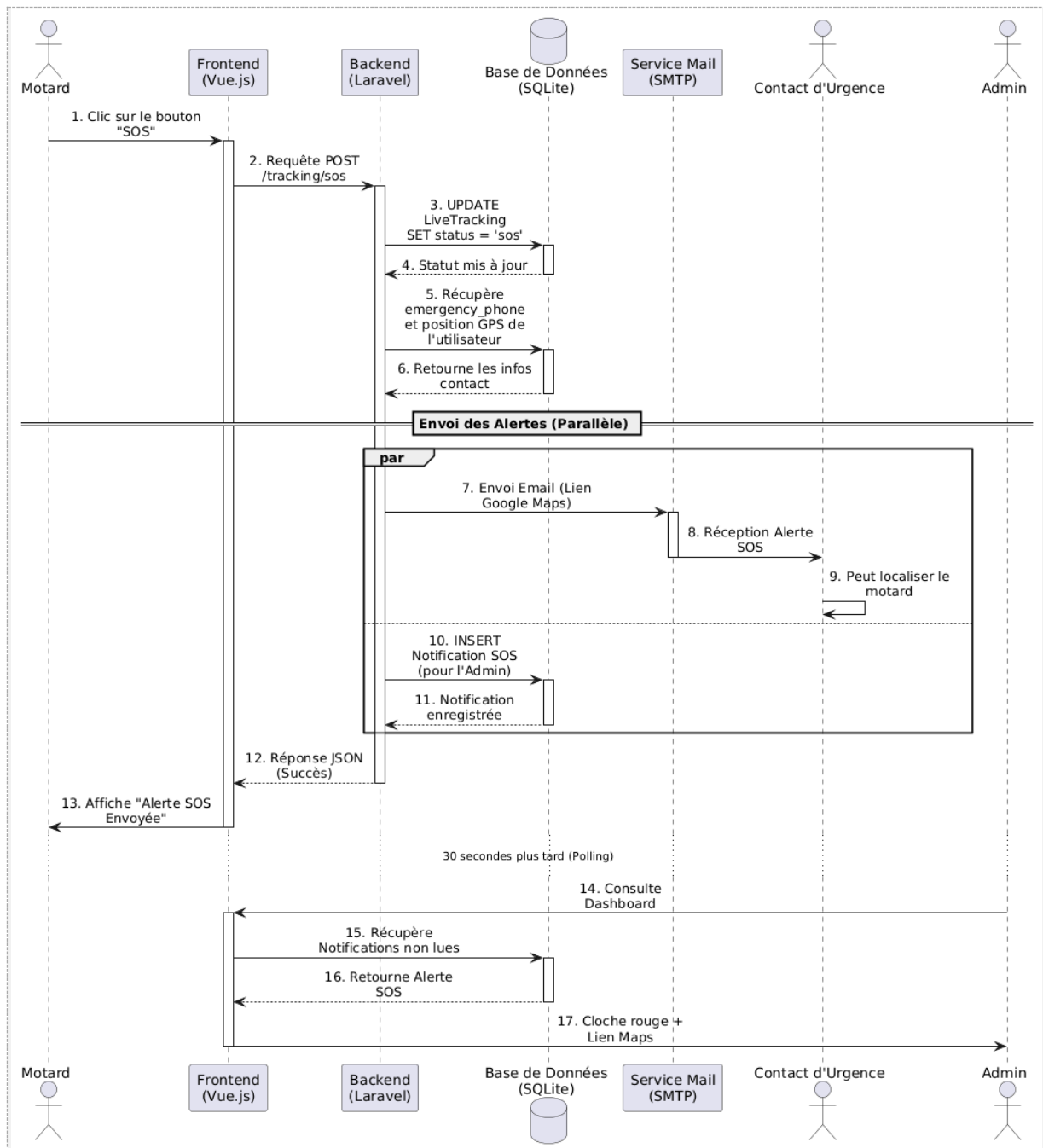
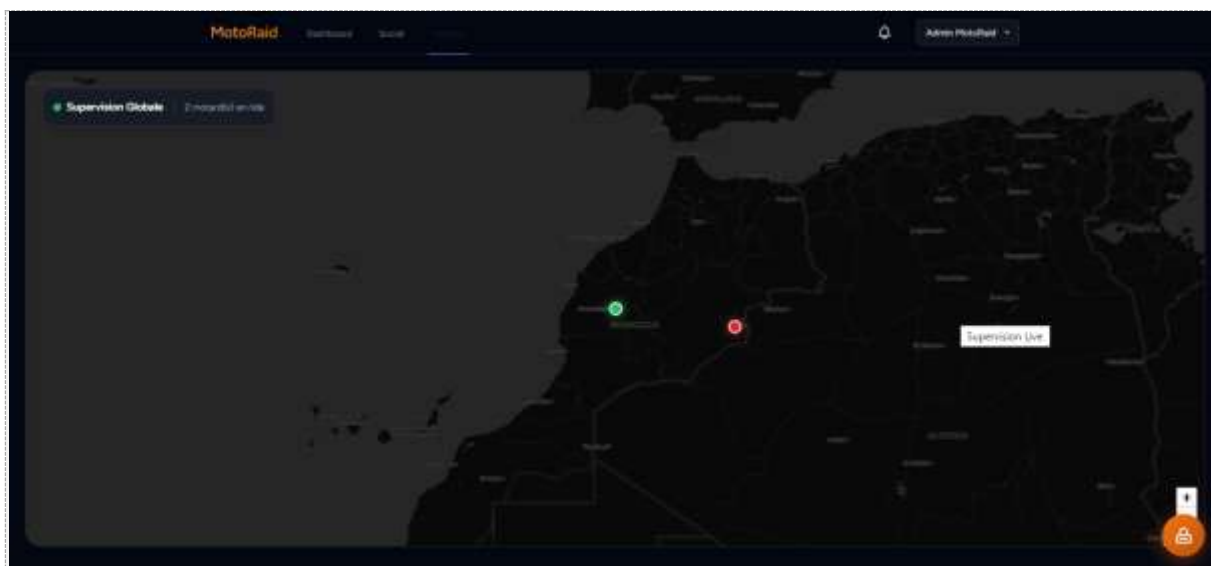


Figure 7 — Diagramme de Séquence du déclenchement SOS

Le diagramme de séquence ci-dessus détaille la chronologie des interactions entre les différents composants du système lors du déclenchement d'une alerte SOS : l'interface utilisateur (client Vue.js), le middleware de sécurité, le contrôleur `SosController`, la classe de service `SosService`, le modèle

AlertSOS, le système de notification de Laravel et, enfin, les deux destinataires de l'alerte que sont le contact d'urgence (par e-mail) et le tableau de bord administrateur (par notification en temps réel).



Capture d'écran 8 — Carte de Tracking Admin et bouton SOS

3.5. Expérience Utilisateur (UX/UI) et Monétisation

Design « Dark Mode / Glassmorphism »

L'identité visuelle de MotoRaid a été conçue autour d'un thème sombre (dark mode) par défaut, choix esthétique cohérent avec l'univers de la moto et favorisant par ailleurs le confort de lecture lors d'une consultation nocturne ou en extérieur. Cette direction artistique s'accompagne d'un style « glassmorphism », caractérisé par des surfaces semi-transparentes à effet de flou (backdrop-blur), des bordures subtiles et des dégradés de couleur, produisant une interface moderne et une hiérarchie visuelle claire des différents panneaux d'information, notamment sur les écrans de cartographie et de tableau de bord où plusieurs couches d'information se superposent.

Dashboard Admin dynamique (Chart.js)

Le tableau de bord administrateur offre une vision synthétique et actualisée de l'activité de la plateforme, à travers un ensemble de graphiques dynamiques réalisés avec la bibliothèque Chart.js : évolution du nombre de réservations, chiffre d'affaires généré, répartition géographique des utilisateurs, taux d'occupation de la flotte par partenaire, nombre de sessions de tracking actives et historique des alertes SOS déclenchées.

Système d'abonnement Freemium et cloche de notifications

La monétisation de la plateforme repose sur un modèle Freemium : l'accès aux fonctionnalités de base (recherche, réservation, planificateur simple) demeure gratuit, tandis qu'un abonnement Premium payant, souscrit via PayPal, débloque des fonctionnalités avancées telles que l'assistant IA, les options étendues du planificateur d'itinéraire ou encore un tracking prolongé. Un système de notifications centralisé, matérialisé par une cloche accessible depuis l'ensemble de l'interface, informe l'utilisateur en temps quasi réel des événements le concernant (confirmation de réservation, nouveau message dans un Ride Hub, validation d'adhésion, rappel d'échéance d'abonnement).

Chapitre 4 : Tests et Déploiement

La qualité logicielle d'une plateforme telle que MotoRaid, qui met en jeu des données sensibles (localisation, alertes d'urgence) et des flux financiers (paiement en ligne), ne saurait être garantie sans une stratégie de test rigoureuse, ni sans une procédure de déploiement maîtrisée. Ce quatrième chapitre en présente les grandes lignes.

4.1. Stratégie de test

Tests manuels

Compte tenu des contraintes temporelles inhérentes à un stage de fin d'études, la stratégie de test adoptée a privilégié une campagne de tests manuels structurée, exécutée à l'issue du développement de chaque module fonctionnel. Chaque parcours utilisateur critique (réservation d'une moto, planification et export d'un itinéraire, création d'un Ride Hub, déclenchement d'une alerte SOS, souscription à un abonnement) a fait l'objet d'un scénario de test détaillé, exécuté tant sur le rôle utilisateur standard que sur les rôles Partenaire et Administrateur, afin de vérifier la bonne application des règles d'autorisation définies au chapitre 2.

Une attention particulière a été portée aux cas limites (edge cases), notamment concernant le calcul de disponibilité des motos (réservations aux dates strictement contiguës, tentative de double réservation simultanée) ainsi qu'à la robustesse du système face à des données d'entrée invalides ou malveillantes, conformément aux principes de validation systématique évoqués précédemment.

Seeders pour les données de démonstration

Afin de faciliter à la fois les phases de test et la présentation du projet à l'encadrant de stage, un ensemble de « seeders » Laravel a été développé. Ces classes permettent de peupler automatiquement la base de données SQLite avec un jeu de données cohérent et réaliste : comptes utilisateurs représentatifs de chaque rôle, catalogue de motos varié réparti entre plusieurs partenaires fictifs, historique de réservations, Ride Hubs actifs avec messages d'exemple, ainsi que des sessions de tracking simulées. Cette approche garantit la reproductibilité de l'environnement de démonstration, indépendamment du poste sur lequel l'application est installée.

4.2. Déploiement

Préparation du livrable

Le livrable final du projet a été préparé de manière à en faciliter l'installation et la prise en main par un tiers, notamment l'encadrant de stage lors de la phase d'évaluation. Cette préparation comprend :

- Un fichier `.env.example` documentant l'ensemble des variables d'environnement nécessaires au fonctionnement de l'application (connexion à la base de données, clés d'API tierces telles qu'Open-Meteo et PayPal Sandbox, paramètres de messagerie sortante) ;
- Un fichier README détaillant, étape par étape, la procédure d'installation (clonage du dépôt, installation des dépendances Composer et npm, génération de la clé d'application, exécution des migrations et des seeders, compilation des assets frontend) ;

- Un script d'installation automatisé, lorsque pertinent, réduisant le nombre de commandes à exécuter manuellement.

Discussion sur l'évolution mobile future (Capacitor.js)

Si la version actuelle de MotoRaid se présente sous la forme d'une application web responsive, pleinement fonctionnelle sur smartphone via le navigateur, une réflexion a été engagée quant à sa transformation future en application mobile native, notamment pour bénéficier d'un accès plus fiable aux capteurs de géolocalisation en arrière-plan, indispensable à un tracking véritablement continu. La solution Capacitor.js, qui permet d'empaqueter une application web existante au sein d'un conteneur natif iOS et Android tout en conservant l'essentiel de la base de code Vue.js déjà développée, a été identifiée comme une piste d'évolution particulièrement pertinente et peu coûteuse en réécriture, et constitue à ce titre l'une des perspectives détaillées en conclusion du présent rapport.

Conclusion Générale

Bilan technique et personnel

Le projet MotoRaid, mené dans le cadre de ce stage de fin d'études, a permis la conception et le développement d'un écosystème numérique complet, couvrant l'ensemble du parcours du motard marocain, de la location d'un véhicule jusqu'à son suivi de sécurité en temps réel, en passant par la planification d'itinéraire et l'animation communautaire. Sur le plan strictement technique, ce stage a constitué une occasion privilégiée de mettre en pratique et d'approfondir des compétences en développement full-stack moderne, à travers la maîtrise conjointe de Laravel 11 et de Vue.js 3, ainsi que de leur articulation par le biais d'Inertia.js, une approche architecturale dont l'efficacité en termes de vélocité de développement a pu être pleinement constatée.

Ce projet a également été l'occasion d'appliquer concrètement des principes de conception logicielle avancés, en particulier les principes SOLID, à travers l'introduction d'une couche de Service Classes venant enrichir le pattern MVC natif de Laravel. Cette démarche, au-delà de sa dimension académique, s'est révélée porteuse d'enseignements très concrets quant à la maintenabilité du code produit, notamment lors des phases de correction de bogues et d'ajout de fonctionnalités tardives, où la localisation et l'isolement de la logique métier concernée se sont trouvés grandement facilités.

Sur le plan personnel, ce stage a permis de développer une compréhension plus fine des enjeux liés à la sécurité applicative (gestion des rôles et des autorisations, validation systématique des données, protection des informations sensibles) ainsi qu'une sensibilité accrue à l'expérience utilisateur, dimension trop souvent négligée dans les projets académiques mais essentielle à l'adoption réelle d'un produit numérique par ses utilisateurs finaux.

Perspectives d'évolution

Plusieurs axes d'évolution ont été identifiés au fil du développement de MotoRaid, dont la mise en œuvre, hors du périmètre temporel de ce stage, constituerait une suite naturelle et cohérente au travail réalisé :

- Migration du mécanisme de messagerie et de tracking en temps réel vers une architecture fondée sur les WebSockets (par exemple via Laravel Reverb ou Pusher), afin de remplacer le polling périodique actuel par une communication bidirectionnelle véritablement instantanée, réduisant à la fois la latence perçue par l'utilisateur et la charge générée sur le serveur ;
- Développement d'une application mobile native, s'appuyant sur Capacitor.js pour capitaliser sur la base de code Vue.js existante, avec un accès renforcé aux capteurs de géolocalisation en arrière-plan et la mise en place de notifications push natives ;
- Enrichissement de l'assistant intelligent intégré à la plateforme, notamment par une meilleure contextualisation de ses réponses au regard de l'historique de réservation et des préférences de l'utilisateur ;
- Passage à un système de gestion de base de données de production (MySQL ou PostgreSQL), en remplacement de la base SQLite utilisée en phase de développement, ainsi que la mise en place d'une infrastructure de déploiement continu (CI/CD) ;

- Extension de la marketplace à d'autres catégories d'équipement (accessoires, équipements de protection individuelle) et internationalisation de la plateforme vers d'autres marchés maghrébins.

En définitive, ce stage aura permis de démontrer la faisabilité technique d'un écosystème numérique intégré au service d'une communauté encore peu digitalisée au Maroc, tout en posant les bases architecturales solides d'un produit dont le potentiel d'évolution demeure, à ce stade, considérable.

Bibliographie / Webographie

La réalisation technique de ce projet s'est appuyée sur les ressources documentaires officielles suivantes :

- Laravel — Documentation officielle du framework, disponible à l'adresse : laravel.com/docs
- Vue.js — Documentation officielle (Composition API), disponible à l'adresse : vuejs.org
- Inertia.js — Documentation officielle, disponible à l'adresse : inertiajs.com
- Tailwind CSS — Documentation officielle, disponible à l'adresse : tailwindcss.com/docs
- Leaflet — Documentation officielle de la bibliothèque cartographique, disponible à l'adresse : leafletjs.com
- OpenStreetMap — Portail des contributeurs et documentation des données cartographiques libres, disponible à l'adresse : openstreetmap.org
- OSRM (Open Source Routing Machine) — Documentation officielle du moteur de routage, disponible à l'adresse : project-osrm.org
- Open-Meteo — Documentation de l'API météorologique gratuite, disponible à l'adresse : open-meteo.com
- Chart.js — Documentation officielle de la bibliothèque de visualisation de données, disponible à l'adresse : chartjs.org
- PayPal Developer — Documentation de l'intégration des paiements et de l'environnement Sandbox, disponible à l'adresse : developer.paypal.com
- Capacitor — Documentation officielle, disponible à l'adresse : capacitorjs.com